TP : apprentissage de représentations auto-supervisé

Master 2 Informatique, parcours VMI - Yanis Hemdane - Code : github.com/Bigadin/teaching_UPC_UFRMI

1. Objectif

Le but du TP est de comparer plusieurs tâches prétextes auto-supervisées. Chaque tâche sert à entraîner un encodeur sans labels. On évalue ensuite la représentation obtenue avec un classifieur linéaire entraîné sur les features gelées. On utilise un classifieur linéaire et pas un MLP pour mesurer la qualité des features elles-mêmes : un classifieur plus puissant pourrait compenser une mauvaise représentation. On part du notebook *selfsupervised_demos* et de la librairie *deepul_helper*, avec les modèles pré-entraînés sur CIFAR10 fournis (context encoder, prédiction de rotation, SimCLR).

2. Comparaison des trois méthodes sur CIFAR10

La fonction *evaluate_accuracy* du notebook ne fait qu'afficher les scores. On utilise donc *evaluate_classifier*, qui renvoie le top-1 et le top-5, dans une boucle sur les trois tâches. Les résultats sont rangés dans un DataFrame pandas puis affichés en tableau et en diagramme en barres (dernières cellules du notebook et script *compare_methods.py*). Sur le test de CIFAR10, on obtient 45,8 % pour le context encoder, 79,9 % pour la rotation et 92,8 % pour SimCLR (top-1).

3. Évaluation sur un autre jeu de données : SVHN

On a choisi SVHN (numéros de maison, 73 257 images d'entraînement et 26 032 de test, 32x32 en couleur, 10 classes). Le format est le même que CIFAR10 mais le domaine est très différent : des chiffres au lieu d'objets. On garde les encodeurs appris sur CIFAR10 et on regarde si leurs features se transfèrent à un autre domaine.

Adaptations faites dans la librairie :

- *data.py* : ajout d'une branche *svhn* dans *get_datasets* et *get_transform*. Le serveur officiel de SVHN ne répondait pas, donc les données ont été récupérées sur le miroir Hugging Face et lues avec une petite classe *SVHNNpz*.
- Transformations : mêmes tailles d'entrée que pour CIFAR10 (128x128 pour le context encoder, 32x32 pour les autres) et même normalisation que CIFAR10, puisque les encodeurs ont été entraînés avec. On retire le flip horizontal : un chiffre retourné n'est plus le même chiffre.
- *demos.py* : *load_model_and_data* utilisait la même variable pour choisir les données, le checkpoint et l'architecture. On a ajouté un paramètre *pretrained_weights* pour charger SVHN avec l'encodeur CIFAR10.
- Le classifieur linéaire du checkpoint prédit les classes de CIFAR10 (avion, chat...) et n'a pas de sens pour SVHN. On en entraîne donc un nouveau (fonction *train_linear_classifier*, Adam, lr = 1e-3, 10 epochs) avec l'encodeur gelé.

4. Nouvelle tâche prétexte : position relative de patchs

On a implémenté la tâche de Doersch et al. (2015) dans *deepul_helper/tasks/relative_position.py*, en suivant la même interface que les autres tâches de la librairie. L'image est agrandie en 64x64 et découpée en une grille 3x3 de cases de 21 pixels. On tire le patch central et un de ses 8 voisins, et le réseau doit prédire la position du voisin (8 classes). Les patchs font 16x16 et sont placés avec un décalage aléatoire dans leur case. Il y a donc toujours un espace entre les deux patchs, ce qui empêche le réseau de répondre juste en prolongeant les bords ou les lignes d'un patch à l'autre.

Les deux patchs passent dans le même encodeur CNN (3 couches de convolution avec BatchNorm, ReLU et max pooling, 256 canaux en sortie). Leurs features sont concaténées puis envoyées à un MLP qui prédit la position. Pour la classification linéaire, on applique l'encodeur à toute l'image et on moyenne la carte de features sur une grille 3x3 (vecteur de 2304 valeurs).

Le modèle a été entraîné 30 epochs sur CIFAR10 (SGD, lr = 0,1 avec cosine annealing, environ 45 min sur une RTX 3060), avec le script *train_relative_position.py*. Comme dans la librairie, un classifieur linéaire est entraîné en même temps sur les features détachées. À la fin, le modèle trouve la bonne position dans environ 55 % des cas (contre 12,5 % au hasard). La précision linéaire sur CIFAR10 passe de 41 % après la première epoch à 66,7 % à la fin.

5. Résultats

Méthode	CIFAR10 top-1	CIFAR10 top-5	SVHN top-1	SVHN top-5
Context encoder	45,8	90,3	62,5	92,8
Rotation	79,9	99,1	79,7	97,9
SimCLR	92,8	99,9	56,6	91,6
Position relative	66,7	96,0	78,3	97,3

Tableau 1 : précision (%) de la classification linéaire sur le jeu de test. Tous les encodeurs sont appris sur CIFAR10.

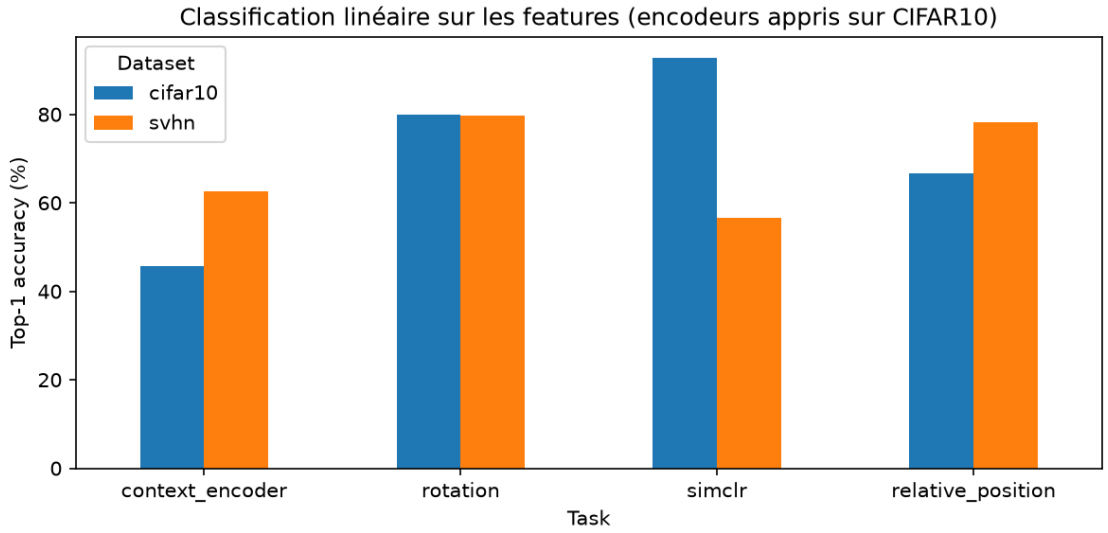


Figure 1 : précision top-1 par méthode, sur CIFAR10 et SVHN.

6. Analyse

Sur CIFAR10, SimCLR est nettement devant, suivi de la rotation, de la position relative et du context encoder. Ce classement est cohérent avec la littérature. La comparaison n'est cependant pas équitable : SimCLR utilise un ResNet-50 entraîné 1000 epochs sur plusieurs GPU, alors que notre modèle de position relative est un petit CNN entraîné 30 epochs. Notre tâche dépasse quand même largement le context encoder, dont la représentation (le goulot de 4000 dimensions d'un auto-encodeur sur des images 32x32 agrandies en 128x128) semble peu adaptée à la classification.

Sur SVHN, le classement change complètement. SimCLR perd 36 points et passe dernier, alors que la rotation reste stable et que la position relative et le context encoder progressent. Notre explication : SimCLR apprend des features très sémantiques et invariantes, adaptées aux objets de CIFAR10 mais pas aux chiffres. Ses augmentations (couleurs, niveaux de gris, crops) le rendent justement insensible à certains détails. Les features de la rotation (couche conv2) et de la position relative sont plus locales : traits, bords, formes. Ces éléments servent aussi pour reconnaître des chiffres. SVHN est aussi un problème plus simple à résoudre linéairement à partir de features bas niveau, ce qui explique que certaines méthodes y fassent mieux que sur CIFAR10.

En résumé, la meilleure méthode sur le domaine d'entraînement n'est pas forcément celle qui se transfère le mieux. Les tâches prétextes géométriques (rotation, position relative) donnent ici des représentations plus générales que l'apprentissage contrastif.

7. Limites

- Les classifieurs CIFAR10 viennent des checkpoints fournis, alors que ceux de SVHN ont été entraînés par nous avec seulement 10 epochs et sans réglage des hyperparamètres. Pour SimCLR, le classifieur n'a pas de normalisation des features et on a gardé des augmentations fortes (ColorJitter, Grayscale), ce qui peut expliquer une partie de sa baisse sur SVHN.
- Une seule exécution par configuration, donc pas d'écart-type. Les différences de 1 ou 2 points (rotation et position relative sur SVHN) ne sont pas significatives.
- Il manque une référence avec un encodeur aléatoire pour mesurer l'apport réel du pré-entraînement. La première epoch de la position relative (41 % sur CIFAR10) en donne une idée approximative.
- Notre tâche n'utilise pas le color dropping de l'article original contre l'aberration chromatique. Il serait intéressant de l'ajouter et d'entraîner plus longtemps.

Remarque : utilisation de l'IA

Un assistant IA (Claude) a été utilisé pendant ce TP pour aider à la rédaction de ce compte rendu, pour régler certains problèmes techniques sans rapport direct avec le sujet (chemins, environnement Python, serveur de SVHN inaccessible, chargement des checkpoints), pour expliquer les notions théoriques (tâches prétextes, évaluation linéaire, transfert de représentation) et pour aider sur le code.